



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Prova Finale in
Ingegneria Informatica

Progetto di Reti Logiche

Author:

Federico Ernesto Bulfari

Student ID:

10837902

Academic Year:

2024-25

Abstract

La seguente è una relazione tecnica sulla creazione di un componente, descritto in linguaggio VHDL ed implementato su una FPGA Artix-7 FPGA xc7a200tfbg484-1, in grado di leggere da una memoria RAM una sequenza di dati, interpretarli, applicare a scelta due tipi di filtri differenziali e salvare i risultati sulla memoria RAM.

Richiesta

Dato un flusso di dati in ingresso, creare un componente in grado di:

- A **start** = '1' iniziare la lettura. Leggere i primi due Byte (K1, K2) dichiaranti la lunghezza della sequenza di valori.
- Leggere il bit meno significativo del terzo Byte, dichiarante il filtro da usare ('0' per il filtro 3, '1' per il filtro 5).
- Leggere altri 14 Byte contenenti i coefficienti da utilizzare (7 per ogni filtro).
- Leggere i valori in ingresso per la lunghezza specificata.
- Applicare la funzione $f'(i) = \frac{1}{n} \cdot \sum C_j \cdot f[j + i]$ ai dati letti.
 - $j \in [-l; +l]$.
 - filtro 3: $l = 2$, filtro 5: $l = 3$.
 - filtro 3: $n = 12$, filtro 5: $n = 60$.
- Scrivere in memoria, alla prima posizione libera dopo la sequenza di valori, i risultati delle operazioni.
- Porre il segnale **done** a '1' a processo concluso. Quando il segnale **start** va a '0' porre **done** a '0'.

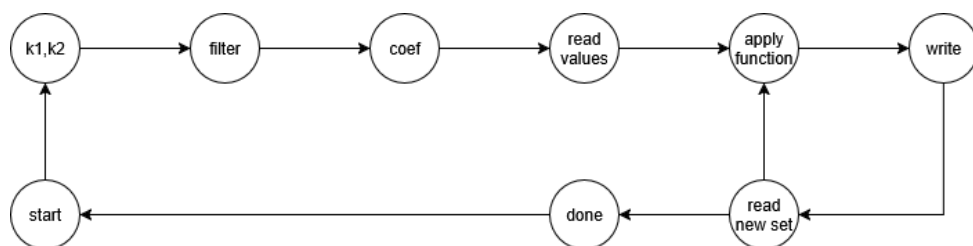


Figura 1: Flusso logico dell'operazione

Indice

Abstract	1
Indice	2
1 Architettura	3
1.1 Modulo FSM (Controller)	3
1.2 Modulo multiplier_radix2	5
1.3 Moduli cla2, cla4, cla8, cla16, cla18	5
1.4 Moduli shift_reg_16 e shift_reg_56	5
1.5 Moduli inc16	5
1.6 Contatori	5
1.7 Atri registri	5
2 Risultati Sperimentali	6
2.1 Sintesi	6
2.2 Simulazioni	7
3 Conclusioni	9

1 | Architettura

L'architettura del sistema è di tipo "controller-datapath": un'unità di controllo centrale, implementata come una Macchina a Stati Finiti (FSM), gestisce il flusso delle operazioni e comanda un datapath.

1.1. Modulo FSM (Controller)

Il Modulo FSM coordina l'intero processo. Questo è implementato da due processi, uno sincrono, `fsm_lambda`, ed uno asincrono, `fsm_delta`, i quali modellano una Macchina a Stati Finiti di Mealy che si compone come illustrato in figura 1.1.

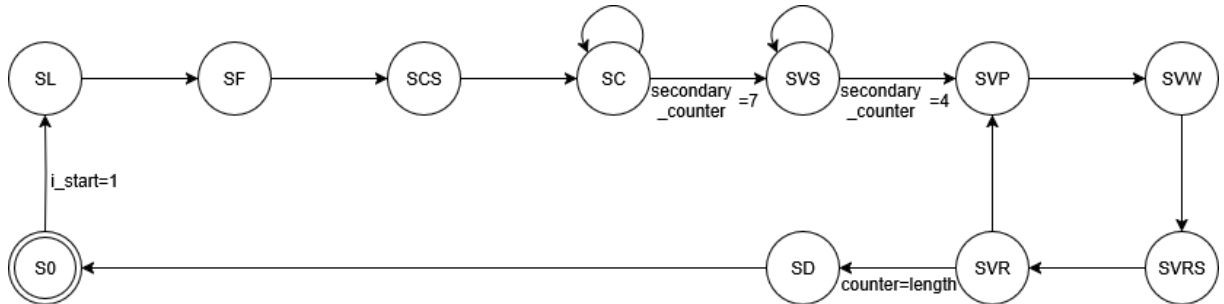


Figura 1.1: Diagramma FSM

State	o_done	o_mem_en	o_mem_we	o_mem_addr	length_reg_en	coef_reg_en	val_reg_en	next_a	next_c	write	start_multiply
S0	0	0	0	addres	0	0	0	0	0	0	0
SL	0	1	0	addres	1	0	0	1	0	0	0
SF	0	1	0	addres	0	0	0	1	0	0	0
SCS	0	1	0	coefficients_pos0	0	1	0	0	1	0	0
SC	0	1	0	coefficients_pos	0	1	0	0	1	0	0
SVS	0	1	0	addres	0	0	1	1	0	0	0
SVP	0	1	0	writer_pointer	0	0	0	0	0	0	1
SVW	0	1	1	writer_pointer	0	0	0	0	0	1	0
SVRS	0	1	0	addres	0	0	0	0	0	0	0
SVR	0	1	0	addres	0	0	1	1	0	0	0
SD	1	0	0	addres	0	0	0	0	0	0	0

Tabella 1.1: Tabella dei valori della FSM lambda

Descrizione degli stati principali

S0 (IDLE) Stato di riposo. Attende il segnale `i_start`. Quando `i_start` è attivo, memorizza l'indirizzo iniziale `i_add` ed inizia la lettura.

SL (Sequence Length) Legge i due Byte che compongono la lunghezza della sequenza di dati da elaborare.

SF (Select Filter) Legge un Byte dalla memoria per determinare se usare il filtro 3 o il filtro 5.

SCS (Coefficients Scan Setup) Calcola la posizione dei coefficienti da leggere in base al filtro scelto, preparando così il componente alla lettura di questi.

SC (Coefficients Scan) Legge sequenzialmente i coefficienti del filtro dalla memoria e li carica nel registro a scorrimento `coef_reg`.

SVS (Values Scan Setup) Carica i primi 4 valori della sequenza di ingresso nel registro `val_reg` per inizializzare la pipeline di calcolo.

SVP (Values Processing) Avvia i 7 moltiplicatori in parallelo per eseguire il prodotto tra i valori e i coefficienti caricati nei registri. Attende il completamento di tutte le moltiplicazioni e ne salva i risultati a 16 bit in `prod_reg`.

I risultati in `prod_reg` vengono sommati assieme da 6 `cla18` e ne viene salvato il risultato in `sum_reg`.

`shifted_output` salva i valori del risultato della somma fratto 16, 64, 256 e 1024. Sommando la frazione per 64 e 1024 è ottenuto `output16_5(filtro 5)` e, sommando i restanti a questo, `output16_3(filtro 3)`. A questi viene sommato rispettivamente 2 e 4 generando così `output16_5_neg` e `output16_3_neg`.

I due output vengono infine compressi a 8 bit valutando prima il segno dell'output scelto dal filtro, quindi se il valore è out of bounds, nel cui caso viene rappresentato con il valore rappresentabile più vicino (-128 o 127). Questo viene salvato su `o_mem_data`.

SVW (Value Write) Lo stato scrive in memoria il contenuto di `o_mem_data` nella prima posizione disponibile dopo i valori letti.

SVRS (Value Read Setup) Prepara il componente per leggere e posizionare un nuovo elemento in `val_reg`, facendo così shiftare i valori in esso contenuti.

SVR (Value Read) Legge il valore successivo dalla sequenza di ingresso, facendolo scorrere all'interno del `val_reg`, e aggiorna i contatori. Il ciclo `SVP` → `SVOC` → `SVW` → `SVR` si ripete per ogni campione di uscita da calcolare. Quando tutti i valori sono stati letti passa allo stato SD.

SD (DONE) Al termine dell'elaborazione di tutta la sequenza, la FSM entra in questo stato, asserisce il segnale `o_done` e si prepara a tornare allo stato S0.

1.2. Modulo multiplier__radix2

Questo modulo implementa un moltiplicatore radix2 a 8 bit. Controllato da una FSM interna, completa la moltiplicazione in 3 cicli di clock, al termine dei quali notifica il completamento con un segnale `done`. Il progetto istanzia 7 di questi moltiplicatori.

1.3. Moduli cla2, cla4, cla8, cla16, cla18

Questi moduli implementano sommatore veloci con architettura Carry-Lookahead (CLA) per minimizzare il ritardo di propagazione del riporto. Il progetto segue un approccio gerarchico: il `cla4` è costruito usando due `cla2`, il `cla8` è costruito usando due `cla4`, il `cla16` è costruito usando due `cla8`, e il `cla18` è costruito usando un `cla16` e un `cla2`.

1.4. Moduli shift_reg_16 e shift_reg_56

Sono registri a scorrimento che interfacciano la memoria (dati a 8 bit per ciclo) con il datapath parallelo. Il `shift_reg_56` accumula 7 Byte per fornire un vettore a 56 bit ai 7 moltiplicatori. Il `shift_reg_16` assembla la lunghezza della sequenza fornita su due Byte.

1.5. Moduli inc16

Questi moduli sono incrementatori utilizzati per aggiornare i puntatori agli indirizzi di memoria RAM e i contatori interni(`counter`,`secondary_counter`). Sono implementati da `cla16`.

1.6. Contatori

`counter` e `secondary_counter` sono due contatori a 16 bit implementati tramite moduli `inc16` che permettono di gestire la quantità di Byte letti da ogni stato.

1.7. Altri registri

Nel componente vengono implementati vari Flip-Flop al fine di memorizzare gli indirizzi di memoria utili e i risultati delle varie operazioni atte al calcolo dell'output.

2 | Risultati Sperimentali

2.1. Sintesi

Analisi di Utilizzo

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	1800	0	0	134600	1.34
LUT as Logic	1800	0	0	134600	1.34
LUT as Memory	0	0	0	46200	0.00
Slice Registers	1038	0	0	269200	0.39
Register as Flip Flop	1038	0	0	269200	0.39
Register as Latch	0	0	0	269200	0.00
F7 Muxes	0	0	0	67300	0.00
F8 Muxes	0	0	0	33650	0.00

Tabella 2.1: Resource Utilization Summary

Come indicato in tabella 2.1, non vi sono Latch, solo Flip-Flop sincronizzati a `i_clk`.

Analisi di Timing

Clock Slack (MET): 4.916 *ns*

2.2. Simulazioni

Testbench 1: Testbench fornito

Verifica: Inizializza una memoria con 24 valori, imposta il filtro 3 (0x00) e fornisce i coefficienti [0,-1,8,0,-8,1,0,1,-9,45,0,-45,9,-1].

Risultato: Come mostrato in figura 2.1, al segnale di start il componente salva l'indirizzo di partenza (i_add) in `adres0` e `adres` ed inizia a leggere i Byte in ingresso. Viene popolato prima lo shift register `length_reg_0`. Viene scelto il filtro memorizzandolo in `chosen_filter` e spostando il puntatore `coef_pos` sul primo coefficiente da leggere. Vengono poi letti i 7 coefficienti del filtro, mentre il puntatore `adres` viene messo sul primo valore da registrare. Letti i coefficienti e salvati in `coef_reg_0`, il componente popola i primi 4 slot di `val_reg_0` con i primi 4 valori a cui applicare il filtro. Il componente quindi inizia un ciclo in cui, per ogni valore della stringa, viene applicato il filtro, viene scritto il risultato e quindi letto un nuovo valore facendo shiftare i valori in `val_reg_0`.

Test Passato in Post-Synthesis Functional Simulation.

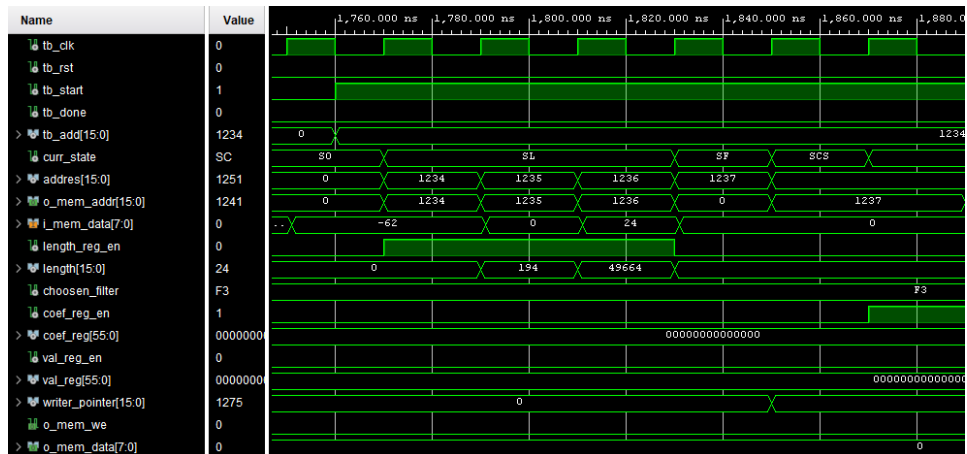


Figura 2.1: Risposta del componente alla salita di i_start

Testbench 2: Letture Consecutive

Verifica: Inizializza una memoria con 24 valori e fornisce i coefficienti [0,-1,8,0,-8,1,0,1,-9,45,0,-45,9,-1]. Al componente viene chiesto di concatenare 2 letture dei valori prima con un filtro 3 (0x00) e poi con un filtro 5 (0x01).

Risultato: Il componente si comporta come prima. Al termine della prima lettura `o_done` viene posto a '1'. Non appena `i_start` torna a '0' `o_done` viene portato a '0', come in figura 2.2. Viene quindi eseguita la seconda lettura usando il filtro 5.

Test Passato in Post-Synthesis Functional Simulation.

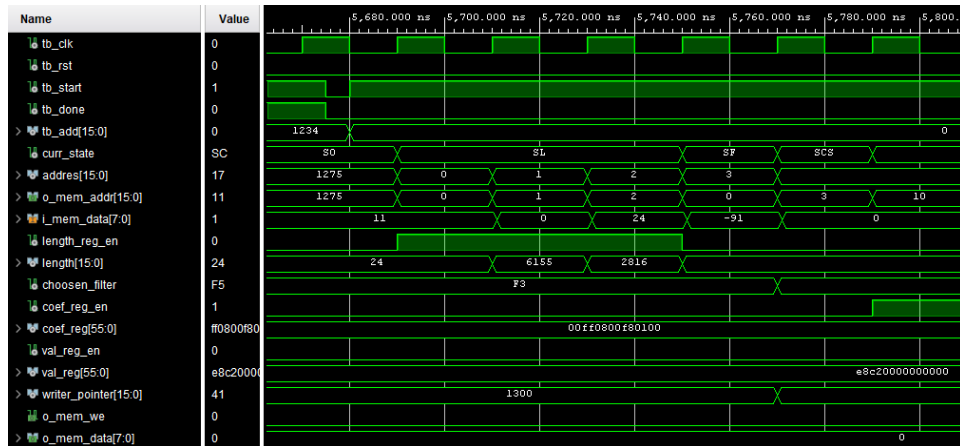


Figura 2.2: Risposta alla caduta di i_start

Testbench 3: Edge Cases

Verifica: Vengono qui testati vari casi limite, come la presenza di valori negli slot 0 e 7 dei coefficienti del filtro 3 o la gestione di valori e coefficienti contemporaneamente pari a -128 o ancora un reset asincrono nel mezzo di un' operazione.

Risultato: Il componente non incontra difficoltà in questi casi limite. I valori in eccesso del filtro 3 sono correttamente scartati, 18 bit delle $cla18$ garantiscono che non vi siano overflow nelle somme ed il complemento a 2 nei moltiplicatori è calcolato su 16 bit, assicurando così di poter correttamente gestire anche il caso $-128 \cdot -128$ come illustrato in figura 2.3.

Test Passato in Post-Synthesis Functional Simulation.

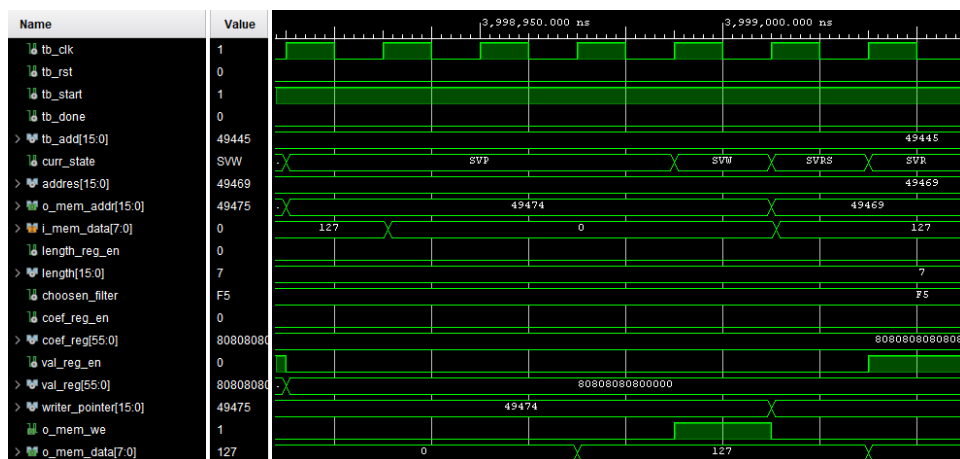


Figura 2.3: Comportamento in caso limite

3 | Conclusioni

Il progetto ha portato allo sviluppo di un componente VHDL con architettura parallela e configurabile dal segnale in ingresso. Le scelte progettuali, come l'uso di moltiplicatori sequenziali e sommatore Carry-Lookahead, mirano a ottimizzare le prestazioni. L'architettura basata su FSM si è rivelata efficace per il controllo. Come già ampiamente discusso, il componente risulta capace di passare tutti i testbench a cui è stato sottoposto.